ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		М.Д. Яушкина
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 7

Тестирование нейронной сети на видеокадрах производственных объектов

по курсу: Системы технического зрения на производстве

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		Г.В. Буренков
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Цель, вариант, объект и классы 4
2 Модель и версия набора 5
3 Подбор порога на valid 6
4 Источник test и нового ролика 8
5 Предобработка и координаты 9
6 Сопоставление рамок 10
7 Метрики test и video_test 12
8 Разбор кадров 13
9 Время обработки 17
10 Оценки по классам 18
11 Файлы результата и режим 20
12 Пригодность модели и следующие опыты 21
ЗАКЛЮЧЕНИЕ 22

ВВЕДЕНИЕ

Работа проверяет модель, выбранную в лабораторной работе №6, на отложенной серии test и на новом ролике. Веса во время теста не меняются. Сначала на valid фиксируется рабочий порог, затем считаются точность, полнота и F1, отдельно по классам, и измеряется время обработки кадра.

1 Цель, вариант, объект и классы

Цель работы — проверить выбранный детектор на видеокадрах и зафиксировать режим, в котором получены оценки. Номер варианта — 1. Объект и классы те же, что в лабораторных работах №5 и №6: шестерня gear, шайба washer и гайка nut. Индивидуальная часть варианта 1 — подсчёт TP, FP, FN, точности, полноты и F1 отдельно по этим трём классам и указание наиболее частой ошибки.

2 Модель и версия набора

Используются веса runs/lr6/v01_b/weights/best.pt, сеть YOLOv8n, вход imgsz = 640. На valid у этого файла mAP@0,5 = 0,995 и mAP@0,5:0,95 = 0,891. Версия набора — 50 кадров 640×480 лабораторной работы №5. Train — серии video01–video03, 30 кадров. Valid — video04, 10 кадров. Test — video05, 10 кадров. Подготовка кадра — копия без отдельного фильтра. Эти сведения совпадают с выбором лабораторной работы №6 и во время теста не меняются.

3 Подбор порога на valid

Рабочий порог уверенности выбирается на valid среди значений 0,15, 0,35 и 0,55. Сначала берётся максимум F1, при равном F1 — большая точность, при равной точности — больший порог. Подавление пересечений NMS выполняется при iou = 0,7. Совпадение с разметкой требует IoU не ниже 0,5 и того же класса. Программа подбора приведена в листинге 1.

Листинг 1 – Подбор порога по valid

```
from pathlib import Path
import json
import torch
from ultralytics import YOLO
from evaluate_frames import evaluate

cfg = json.loads(Path("results/selected_model.json").read_text(encoding="utf-8"))
model = YOLO(cfg["weights"])
device = 0 if torch.cuda.is_available() else "cpu"
rows = []
for conf in (0.15, 0.35, 0.55):
    result = evaluate(model, "dataset/valid/images",
                      "dataset/valid/labels", conf,
                      cfg["imgsz"], device)
    if result["F1"] is None:
        raise ValueError("В valid нет размеченных объектов")
    rows.append(dict(conf=conf, **result))
best = max(rows, key=lambda row: (row["F1"], row["P"], row["conf"]))
cfg.update(conf=best["conf"], nms_iou=0.7, match_iou=0.5)
Path("results/lr7_mode.json").write_text(
    json.dumps(cfg, ensure_ascii=False, indent=2), encoding="utf-8")
Path("results/lr7_thresholds.json").write_text(
    json.dumps(rows, ensure_ascii=False, indent=2), encoding="utf-8")
print("Выбранный порог:", best["conf"])
```

Три порога дали одинаковые суммы на valid. Результат приведён в таблице 1. На десяти кадрах 21 объект разметки: 20 верных обнаружений, ни одной лишней рамки и один пропуск. Точность равна 1, полнота — 0,952, F1 — 0,976. По правилу равенства выбран больший порог 0,55.

Таблица 1 – Подбор порога на valid

Порог	**TP**	**FP**	**FN**	**P**	**R**	**F1**
0,15	20	0	1	1,000	0,952	0,976
0,35	20	0	1	1,000	0,952	0,976
0,55	20	0	1	1,000	0,952	0,976

Единственная ошибка valid — пропуск гайки на кадре video04_0003, уже разобранный при выборе модели. Лишних рамок нет даже при пороге 0,15, поэтому снижение порога полноту не поднимает. Зафиксированный

режим: conf = 0,55, iou NMS = 0,7, порог IoU сопоставления = 0,5, imgsz = 640, веса runs/lr6/v01_b/weights/best.pt. Дальше порог не подстраивается по test и по новому ролику.

4 Источник test и нового ролика

Выборка test — серия video05 набора лабораторной работы №5, 10 кадров 640×480. Она не входила в train и valid. В ней 26 объектов: шестерня — 10, шайба — 8, гайка — 8.

Новый ролик записан отдельной серией тех же трёх деталей. Длительность 22 с, частота 25 кадров/с, размер 640×480, всего 550 кадров. Файл ролика — data/video/test_video.mp4. Детали движутся, меняются масштаб и яркость. На участке около 10–14 с шестерня перекрывает гайку. Участок 18–22 с пустой. Для оценки взяты 30 кадров из разных мест ролика, разметка рамок следует тем же правилам, что и набор лабораторной работы №5. Кадры ролика в train и valid не входят.

В 30 размеченных кадрах ролика 72 объекта: по 24 шестерни, шайбы и гайки. Шесть кадров пустые: test_video_000454, 000473, 000492, 000511, 000530 и 000549. Тридцать кадров принадлежат одному ролику. Это одна серия условий, а не тридцать независимых производственных ситуаций.

5 Предобработка и координаты

К каждому кадру применяется та же функция, что и при согласовании разметки. Она возвращает копию BGR того же размера. Ролик уже имеет размер 640×480, поэтому геометрия не меняется. Листинг 2 повторяет функцию.

Листинг 2 – Подготовка кадра перед детектором

```
def prepare_frame(frame):
    prepared = frame.copy()
    # Разметка ЛР5 стоит на кадрах 640x480 без отдельного фильтра.
    return prepared
```

Разметка хранится в нормализованных координатах YOLO: центр и размеры рамки относительно ширины и высоты кадра. Перед сопоставлением они переводятся в пиксели подготовленного кадра. Предсказание возвращает углы рамки x1, y1, x2, y2 в той же пиксельной системе. Эти углы записываются в detections.csv. Если бы подготовка меняла размер, сопоставление с разметкой было бы неверным. Здесь ширина и высота до и после подготовки совпадают: 640 и 480.

Листинг 3 – Перевод разметки в пиксели кадра

```
def read_truth(path, width, height, n_classes):
    truth = []
    for line in path.read_text(encoding="utf-8").splitlines():
        if not line.strip():
            continue
        parts = line.split()
        if len(parts) != 5:
            raise ValueError(f"Нужна рамочная разметка: {path}")
        cls = int(parts[0])
        xc, yc, bw, bh = map(float, parts[1:])
        if not 0 <= cls < n_classes or min(bw, bh) <= 0:
            raise ValueError(f"Некорректная разметка: {path}")
        box = [(xc - bw / 2) * width, (yc - bh / 2) * height,
               (xc + bw / 2) * width, (yc + bh / 2) * height]
        truth.append((cls, box))
    return truth
```

6 Сопоставление рамок

Предсказания сортируются по убыванию уверенности. Каждое предсказание сопоставляется со свободной рамкой разметки того же класса, у которой наибольший IoU. Если этот IoU не ниже 0,5, пара считается верным обнаружением TP, и рамка разметки больше не используется. Предсказание без пары — лишняя рамка FP. Рамка разметки без пары — пропуск FN. Одна рамка разметки даёт не больше одного TP. Точность равна TP / (TP + FP), а при отсутствии предсказаний равна 0. Полнота и F1 определены, когда в выборке есть объекты разметки. На test, valid и размеченных кадрах ролика объекты есть. Суммы TP, FP и FN сначала собираются по кадрам, и только потом из сумм считаются P, R и F1. Среднее по кадрам не используется. Листинг 4 содержит расчёт IoU, подсчёт и перевод в оценки.

Листинг 4 – Сопоставление рамок и оценки

```
def iou(a, b):
    width = max(0.0, min(a[2], b[2]) - max(a[0], b[0]))
    height = max(0.0, min(a[3], b[3]) - max(a[1], b[1]))
    intersection = width * height
    area_a = max(0.0, a[2] - a[0]) * max(0.0, a[3] - a[1])
    area_b = max(0.0, b[2] - b[0]) * max(0.0, b[3] - b[1])
    union = area_a + area_b - intersection
    return intersection / union if union > 0 else 0.0


def match_counts(truth, predicted, match_iou=0.5):
    used = set()
    tp = 0
    for cls, confidence, box in sorted(
            predicted, key=lambda item: item[1], reverse=True):
        candidates = [(iou(box, other), index)
                      for index, (label, other) in enumerate(truth)
                      if index not in used and label == cls]
        overlap, index = max(candidates, default=(0.0, -1))
        if index >= 0 and overlap >= match_iou:
            used.add(index)
            tp += 1
    return tp, len(predicted) - tp, len(truth) - tp


def scores(tp, fp, fn):
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else None
    f1 = 2 * tp / (2 * tp + fp + fn) if tp + fn else None
    return dict(TP=tp, FP=fp, FN=fn, P=precision, R=recall, F1=f1)
```

Проверка на трёх простых случаях дала ожидаемые тройки (TP, FP, FN). Один совпавший прямоугольник: (1, 0, 0). Два предсказания того же класса на одну рамку: (1, 1, 0). Предсказание другого класса на ту же рамку:

(0, 1, 1). Полученные значения совпали с ожидаемыми: (1, 0, 0), (1, 1, 0), (0, 1, 1).

7 Метрики test и video_test

Оценки test при пороге 0,55 приведены в таблице 2. Десять кадров, 26 объектов разметки, 23 верных обнаружения, лишних рамок нет, три пропуска. Точность равна 1, полнота — 0,885, F1 — 0,939. Отдельная проверка best.pt на test даёт mAP@0,5 = 0,961 и mAP@0,5:0,95 = 0,873. В расчёте mAP порог уверенности другой, поэтому точность и полнота из этой проверки в таблицу рабочего порога не входят.

Таблица 2 – Test при пороге 0,55

Кадры	**Объекты**	**TP**	**FP**	**FN**	**P**	**R**	**F1**
10	26	23	0	3	1,000	0,885	0,939

Для test mAP@0,5 = 0,961 и mAP@0,5:0,95 = 0,873. Оценки 30 кадров ролика при том же пороге приведены в таблице 3. Объектов 72, верных обнаружений 69, лишних рамок 2, пропусков 3. Точность равна 0,972, полнота — 0,958, F1 — 0,965.

Таблица 3 – Кадры ролика при пороге 0,55

Кадры	**Объекты**	**TP**	**FP**	**FN**	**P**	**R**	**F1**
30	72	69	2	3	0,972	0,958	0,965

Шесть пустых кадров ролика не дали ни одной рамки. Они не увеличили ни FP, ни FN. Разбор по классам и наиболее частая ошибка приведены в разделе 10.

8 Разбор кадров

Цвета те же, что при разборе valid. Зелёная рамка совпала с разметкой своего класса, оранжевая — пропуск, красная — лишняя рамка. Показаны верное обнаружение, пропуски, лишняя рамка и пустой участок.

На рисунке 1 первый размеченный кадр ролика, test_video_000000. Шестерня, гайка и шайба найдены, уверенность около 1,00, 0,99 и 0,97. Лишних рамок нет.

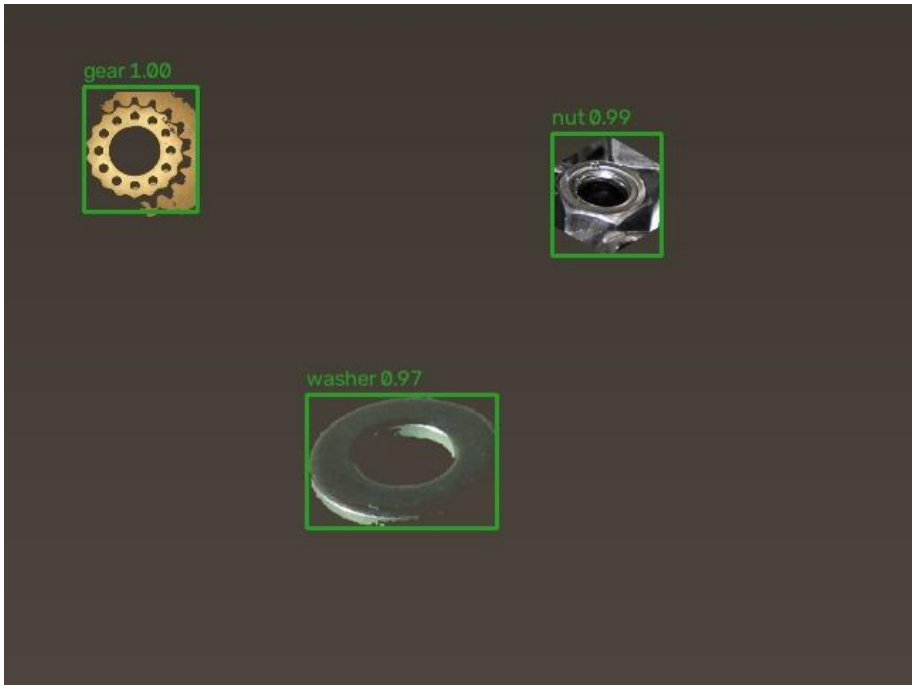


Рисунок 1 – Кадр test_video_000000: три класса найдены

На рисунке 2 кадр test video05_0002. Гайка и шестерня найдены, уверенность около 1,00 и 0,98. Шайба пропущена: шестерня лежит на кольце шайбы, и кольцевой контур в перекрытии не удерживается отдельной рамкой.



Рисунок 2 – Кадр video05_0002: шайба под шестерней пропущена

На рисунке 3 кадр ролика test_video_000265, около 10,6 с. Шестерня и шайба найдены, уверенность около 0,97 и 0,95. Гайка пропущена, её площадь около 0,019 кадра, она закрыта шестерней. Такой же пропуск гайки есть на кадре test_video_000246. На test тот же вид ошибки дают кадры video05_0001 и video05_0009: остаются шестерня и шайба, гайка в перекрытии пропадает.

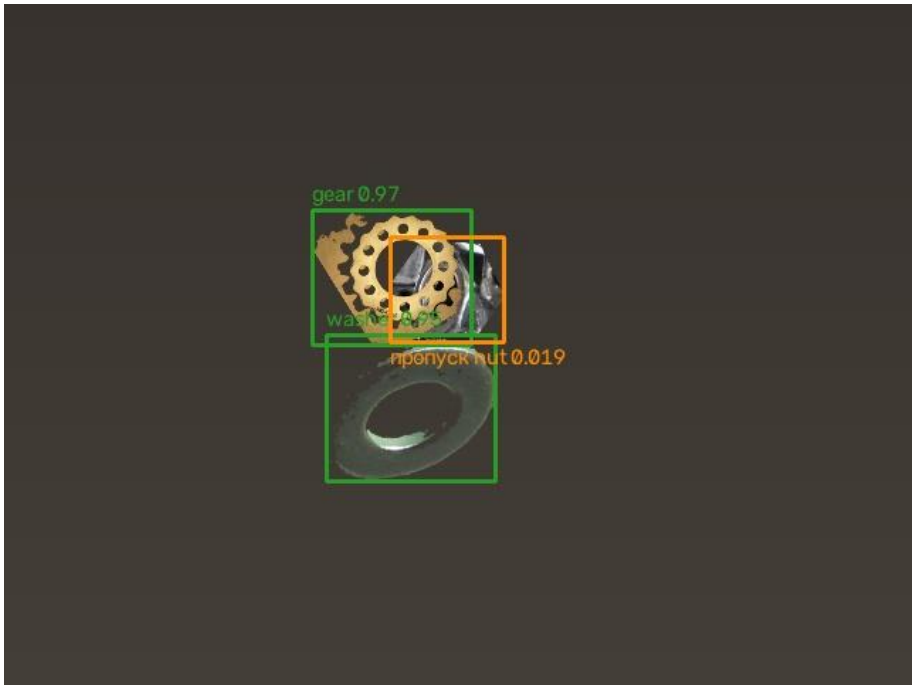


Рисунок 3 – Кадр test_video_000265: гайка под шестерней пропущена

На рисунке 4 кадр test_video_000416, около 16,6 с. Все три объекта разметки найдены. Дополнительно появилась вторая рамка гайки с уверенностью около 0,72 на перекрытии шестерни и гайки. Это лишняя рамка: TP = 3, FP = 1, FN = 0.

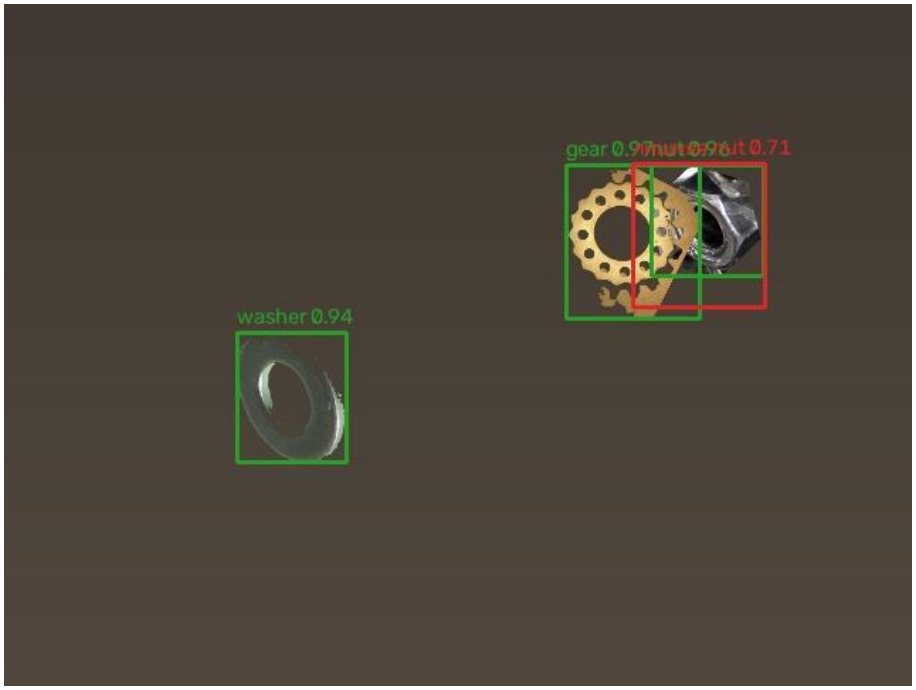


Рисунок 4 – Кадр test_video_000416: лишняя рамка гайки

На рисунке 5 кадр test_video_000435. Шайба и одна шестерня найдены. Вторая рамка шестерни с уверенностью около 0,76 не совпала с разметкой шестерни и осталась лишней. Гайка пропущена. Отдельного совпадения рамки чужого класса с чужой разметкой нет: ошибка учитывается как FP класса gear и FN класса nut. По контуру это тот же случай, что и пропуск гайки под зубчатым контуром, плюс вторая рамка на зоне перекрытия.

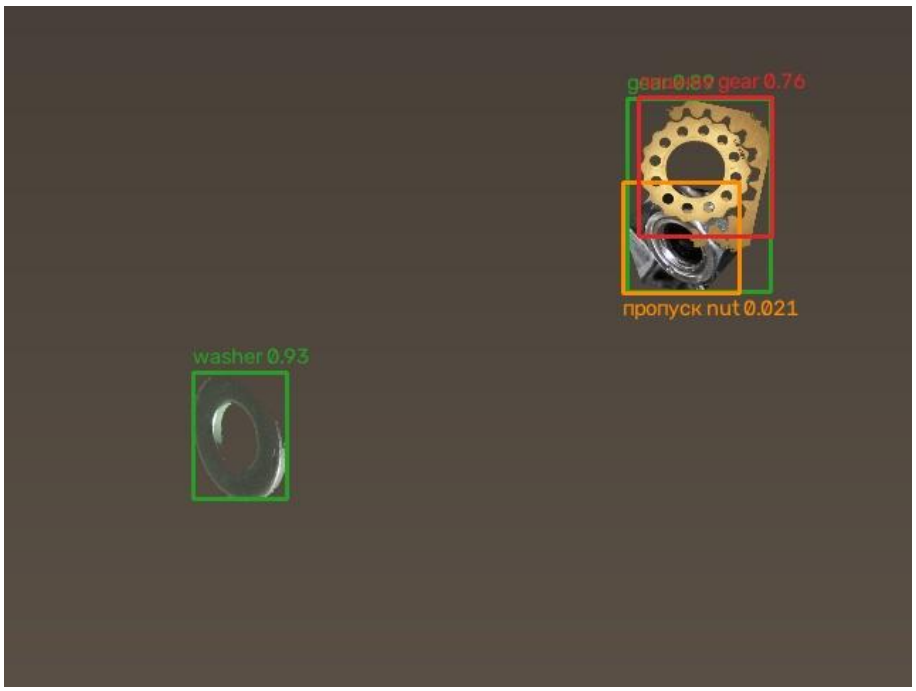


Рисунок 5 – Кадр test_video_000435: пропуск гайки и лишняя шестерня

На рисунке 6 кадр test_video_000454 пустого участка. Объектов в разметке нет, детектор рамку не поставил. Так же ведут себя остальные пять пустых кадров.



Рисунок 6 – Кадр test_video_000454: пустой участок без рамок

9 Время обработки

Время измерено на всех 550 кадрах ролика при imgsz = 640, conf = 0,55 и устройстве CUDA 0. Ускоритель — NVIDIA GeForce RTX 2060, 6 ГБ. Перед циклом выполнены пять вызовов predict на первом кадре, они в таблицу не входят. Полный цикл кадра включает чтение кадра, подготовку, predict, отрисовку, запись кадра в видео и строки таблицы обнаружений. Закрытие видеофайла и сброс кодека после цикла в замер не входят. Пустые кадры в замере есть.

Частота сохранённого annotated.mp4 равна частоте источника, 25 кадров/с. Это частота воспроизведения файла. Она сама по себе не показывает скорость обработки. Скорость берётся из timings.csv. Бюджет одного кадра при 25 кадрах/с равен 40 мс. Результаты приведены в таблице 4.

Таблица 4 – Время обработки ролика

Показатель	**Значение**
Кадров	550
Среднее время predict, мс	10,45
Среднее время цикла, мс	12,58
Медиана цикла, мс	12,38
95-й процентиль цикла, мс	15,10
Производительность, кадров/с	79,5
Бюджет кадра, мс	40
Доля циклов дольше бюджета, %	0

Средний цикл 12,58 мс короче бюджета 40 мс. Доля циклов длиннее бюджета равна нулю. 95-й процентиль 15,10 мс тоже лежит ниже 40 мс. Замер относится к этому ролику, этому входу и включённой записи видео.

10 Оценки по классам

Для варианта 1 суммы TP, FP и FN считаются отдельно по классам тем же правилом сопоставления. Фрагмент накопления приведён в листинге 5. Из сумм класса получаются его точность, полнота и F1.

Листинг 5 – Накопление TP, FP и FN по классам

```
                    boxes.xyxy.tolist())]
    for cls in counts:
        gt = [(c, b) for c, b in truth if c == cls]
        pr = [(c, s, b) for c, s, b in predicted if c == cls]
        values = match_counts(gt, pr, match_iou)
        counts[cls] = [a + b for a, b in zip(counts[cls], values)]
total = [sum(values[i] for values in counts.values())
         for i in range(3)]
per_class = {model.names[cls]: scores(*values)
             for cls, values in counts.items()}
```

На test при пороге 0,55 результат по классам приведён в таблице 5. Шестерня найдена полностью: 10 из 10. У шайбы один пропуск, у гайки два. Лишних рамок ни у одного класса нет.

Таблица 5 – Классы на test при пороге 0,55

Класс	**TP**	**FP**	**FN**	**P**	**R**	**F1**
gear	10	0	0	1,000	1,000	1,000
washer	7	0	1	1,000	0,875	0,933
nut	6	0	2	1,000	0,750	0,857

На 30 кадрах ролика результат приведён в таблице 6. Шайба найдена полностью, 24 из 24, без лишних рамок. У шестерни одна лишняя рамка и ни одного пропуска: точность 0,960, полнота 1, F1 — 0,980. У гайки 21 верное обнаружение, одна лишняя рамка и три пропуска: точность 0,955, полнота 0,875, F1 — 0,913.

Таблица 6 – Классы на кадрах ролика при пороге 0,55

Класс	**TP**	**FP**	**FN**	**P**	**R**	**F1**
gear	24	1	0	0,960	1,000	0,980
washer	24	0	0	1,000	1,000	1,000
nut	21	1	3	0,955	0,875	0,913

Наиболее частая ошибка — пропуск гайки, когда шестерня её закрывает. На test это два пропуска из трёх, кадры video05_0001 и

video05_0009. Третий пропуск test — шайба под шестерней на video05_0002. На ролике все три пропуска относятся к гайке, кадры test_video_000246, 000265 и 000435. Две лишние рамки тоже стоят на перекрытии: вторая гайка на 000416 и вторая шестерня на 000435. Отдельной ошибки «рамка класса совпала с разметкой другого класса» нет. Труднее всего обнаруживается гайка.

11 Файлы результата и режим

Сохранены три файла результата. Видео с рамками — results/lr7_video/annotated.mp4, частота записи равна 25 кадрам/с источника. Таблица обнаружений — results/lr7_video/detections.csv, столбцы frame, time_s, class, conf, x1, y1, x2, y2. Таблица времени — results/lr7_video/timings.csv, столбцы frame, time_s, predict_ms, cycle_ms, objects.

Режим, при котором получены эти файлы: веса runs/lr6/v01_b/weights/best.pt, imgsz = 640, conf = 0,55, iou NMS = 0,7, порог IoU сопоставления = 0,5, устройство CUDA 0 на NVIDIA GeForce RTX 2060, подготовка кадра без изменения размера. Порог взят по valid и на test и на ролике не менялся.

12 Пригодность модели и следующие опыты

В исследованных условиях модель пригодна для отделения трёх деталей на кадрах без сильного перекрытия. На test F1 равен 0,939 при точности 1. На размеченных кадрах ролика F1 равен 0,965. Пустой участок ложных рамок не дал. Средний цикл обработки 12,58 мс и 95-й процентиль 15,10 мс оба ниже бюджета 40 мс этого ролика.

Этого недостаточно, чтобы считать детектор готовым для любых производственных условий. Набор обучения содержит 50 кадров одной постановки, а 30 кадров проверки принадлежат одному ролику. Наиболее частая ошибка — пропуск закрытой гайки. Следующий опыт стоит посвятить примерам перекрытия и проверке, что частично скрытая гайка не поглощается рамкой шестерни. Порог, веса и вход при таком опыте нужно зафиксировать до просмотра новой тестовой серии.

ЗАКЛЮЧЕНИЕ

Проверена модель лабораторной работы №6 с входом 640 и весами runs/lr6/v01_b/weights/best.pt. По valid зафиксирован порог уверенности 0,55. На test F1 равен 0,939, mAP@0,5:0,95 равен 0,873. На 30 кадрах нового ролика F1 равен 0,965. Средний цикл 12,58 мс и 95-й процентиль 15,10 мс ниже бюджета кадра 40 мс. Наиболее частая ошибка — пропуск гайки под шестерней.